Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Структури даних та алгоритми

Лабораторна робота №2

«Алгоритми з вкладеними циклами та метод динамічного програмування»

Виконав:
студент групи ІО-61
Астаф’єв Олександр Владиславович
Номер у списку групи: 2
Перевірив:
Русінов Володимир Володимирович

Київ – 2026

Постанова задачі:

1. Задане натуральне число n. Обчислити значення заданої за варіантом формули (ряду або добутку, що наближено обчислює значення певної функції у заданій точці; точність наближення залежить від кількості ітерацій n).
2. Для розв'язання задачі написати **дві програми**:
   i. перша програма обчислює формулу за допомогою **вкладених циклів**;
   ii. друга програма обчислює ту саму формулу за допомогою **одного циклу** з використанням **методу динамічного програмування** (тобто накопичуючи черговий доданок/множник на основі попереднього, без повторного обчислення статечних, факторіальних і подібних виразів «з нуля» на кожній ітерації).
3. Виконати розрахунок кількості операцій для кожного з алгоритмів за методикою, викладеною на лекції, додавши до неї підрахунок кількості викликів стандартних функцій.
4. Програма має правильно розв'язувати задачу при будь-якому заданому n, для якого результат обчислення коректно представляється типом double.
5. Результат виводити у форматі із **сімома знаками після коми**.

$$2)\ P = \cos x\ = 0.5 \prod_{i=-n}^{n} \frac{2x - \sum_{j=1}^{i} \pi}{x + \sum_{j=1}^{i} \pi} \quad .$$

Текст програми 1:

```
#include <math.h>

#include <stdio.h>
```

```
#define PI 3.14159265358979323846

int main() {
  double x;
  printf("Enter x: ");
  scanf("%lf", &x);

  int n;
  printf("Enter n: ");
  scanf("%d", &n);

  int op_c = 0;

  op_c += 1;
  if (x == 0.0) {
    printf("cos(0) = 1.0000000\n");
    return 0;
  }

  double P = 1;
  op_c += 1;

  double two_x = 2 * x;
  op_c += 2;
```

```
op_c += 2;
for (int i = -n; i <= n; i++) {
  op_c += 1;

  double sum = 0;
  op_c += 1;

  op_c += 1;
  if (i > 0) {
    op_c += 1;
    for (int j = 1; j <= i; j++) {
      op_c += 1;
      sum += PI;
      op_c += 2;
      op_c += 1;
    }
    op_c += 1;
  } else {
    op_c += 2;
    for (int j = i + 1; j <= 0; j++) {
      op_c += 1;
      sum -= PI;
      op_c += 2;
      op_c += 1;
    }
    op_c += 1;
```

```
  }

  double numerator = two_x - sum;
  op_c += 2;
  double denominator = x + sum;
  op_c += 2;

  P *= numerator / denominator;
  op_c += 3;

  op_c += 1;
}
op_c += 1;

P *= 0.5;
op_c += 2;

double lib = cos(x);
op_c += 2;
double delta = fabs(P - lib);
op_c += 3;

printf("P: %.7lf\n", P);
printf("cos(x) (library): %.7lf\n", lib);
printf("delta: %.7lf\n", delta);
printf("Number of operation: %d\n", op_c);
```

```
  return 0;
}
```

Результати тестування програми 1:

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task1.exe
Enter x: 0.5
Enter n: 3
P: 0.8967482
cos(x) (library): 0.8775826
delta: 0.0191656
Number of operation: 157
```

Input interpretation:

$$0.5 \prod_{i=-3}^{3} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

Result:

0.896748

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task1.exe
Enter x: 0.5
Enter n: 10
P: 0.8839533
cos(x) (library): 0.8775826
delta: 0.0063707
Number of operation: 738
```

Input interpretation:

$$0.5 \prod_{i=-10}^{10} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

Result:

0.883953

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task1.exe
Enter x: 0.5
Enter n: 30
P: 0.8797716
cos(x) (library): 0.8775826
delta: 0.0021891
Number of operation: 4558
```

Input interpretation:

$$0.5 \prod_{i=-30}^{30} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

Result:

0.879772

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task1.exe
Enter x: 0.5
Enter n: 100
P: 0.8782464
cos(x) (library): 0.8775826
delta: 0.0006638
Number of operation: 43128
```

Input interpretation:

$$0.5 \prod_{i=-100}^{100} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

Result:

0.878246

Текст програми 2:

```
#include <math.h>
```

```
#include <stdio.h>

#define PI 3.14159265358979323846

int main() {
  double x;
  printf("Enter x: ");
  scanf("%lf", &x);

  int n;
  printf("Enter n: ");
  scanf("%d", &n);

  int op_c = 0;

  op_c += 1;
  if (x == 0.0) {
    printf("cos(0) = 1.0000000\n");
    return 0;
  }

  double P = 1;
  op_c += 1;

  double two_x = 2 * x;
  op_c += 2;
```

```
double t = -n * PI;
op_c += 3;

op_c += 2;
for (int i = -n; i <= n; i++) {
  op_c += 1;

  double numerator = two_x - t;
  op_c += 2;
  double denominator = x + t;
  op_c += 2;

  P *= numerator / denominator;
  op_c += 3;

  t += PI;
  op_c += 2;

  op_c += 1;
}
op_c += 1;

P *= 0.5;
op_c += 2;
```

```
  double lib = cos(x);
  op_c += 2;
  double delta = fabs(P - lib);
  op_c += 3;

  printf("P: %.7lf\n", P);
  printf("cos(x) (library): %.7lf\n", lib);
  printf("delta: %.7lf\n", delta);
  printf("Number of operation: %d\n", op_c);

  return 0;
}
```

Результати тестування програми 2:

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task2.exe
Enter x: 0.5
Enter n: 2
P: 0.9044060
cos(x) (library): 0.8775826
delta: 0.0268235
Number of operation: 72
```

Input interpretation:

$$0.5 \prod_{i=-2}^{2} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

Result:

0.904406

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task2.exe
Enter x: 0.5
Enter n: 20
P: 0.8808412
cos(x) (library): 0.8775826
delta: 0.0032586
Number of operation: 468
```

Input interpretation:

$$0.5 \prod_{i=-20}^{20} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

Result:

0.880841

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task2.exe
Enter x: 0.5
Enter n: 50
P: 0.8789041
cos(x) (library): 0.8775826
delta: 0.0013215
Number of operation: 1128
```

Input interpretation:

$$0.5 \prod_{i=-50}^{50} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

Result:

0.878904

```
PS D:\Astafiev\АСД\sda-astafiev\Lab2> .\task2.exe
Enter x: 0.5
Enter n: 100
P: 0.8782464
cos(x) (library): 0.8775826
delta: 0.0006638
Number of operation: 2228
```

Input interpretation:

$$0.5 \prod_{i=-100}^{100} \frac{2 \times 0.5 - i\pi}{0.5 + i\pi}$$

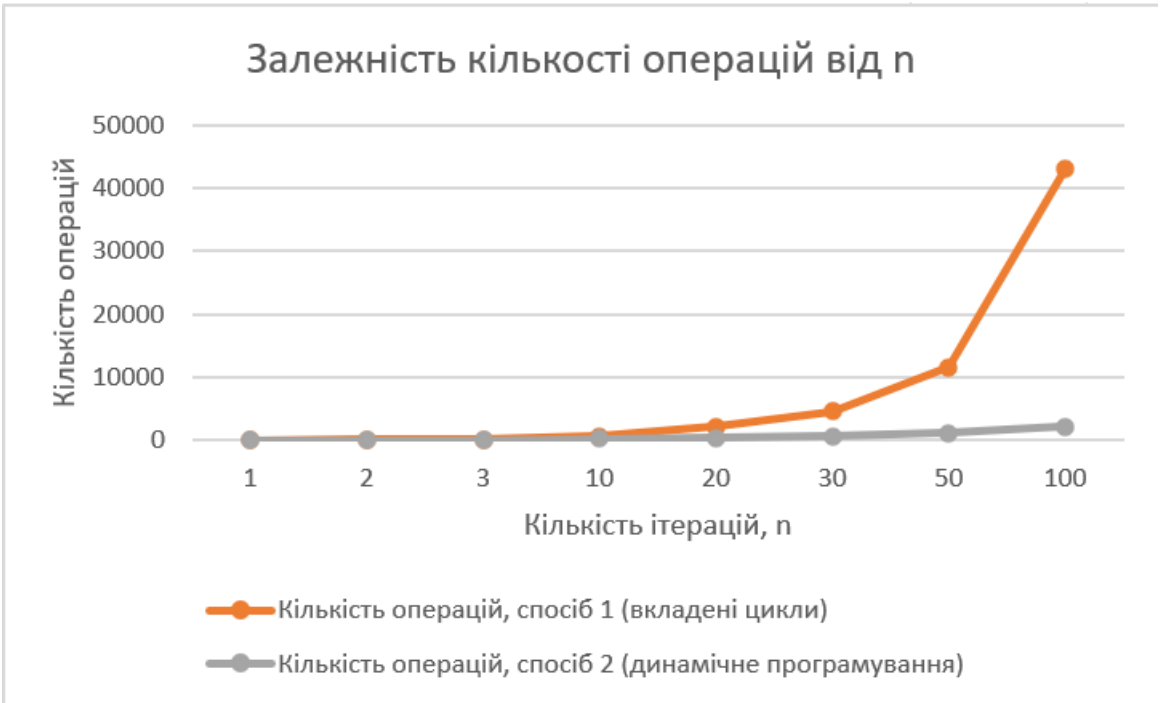
Result:

0.878246

Таблиця результатів:

n	1	2	3	10	20	30	50	100
Кількість операцій, спосіб 1 (вкладені цикли)	63	106	157	738	2248	4558	11578	43128
Кількість операцій, спосіб 2 (динамічне програмування)	50	72	94	248	468	688	1128	2228

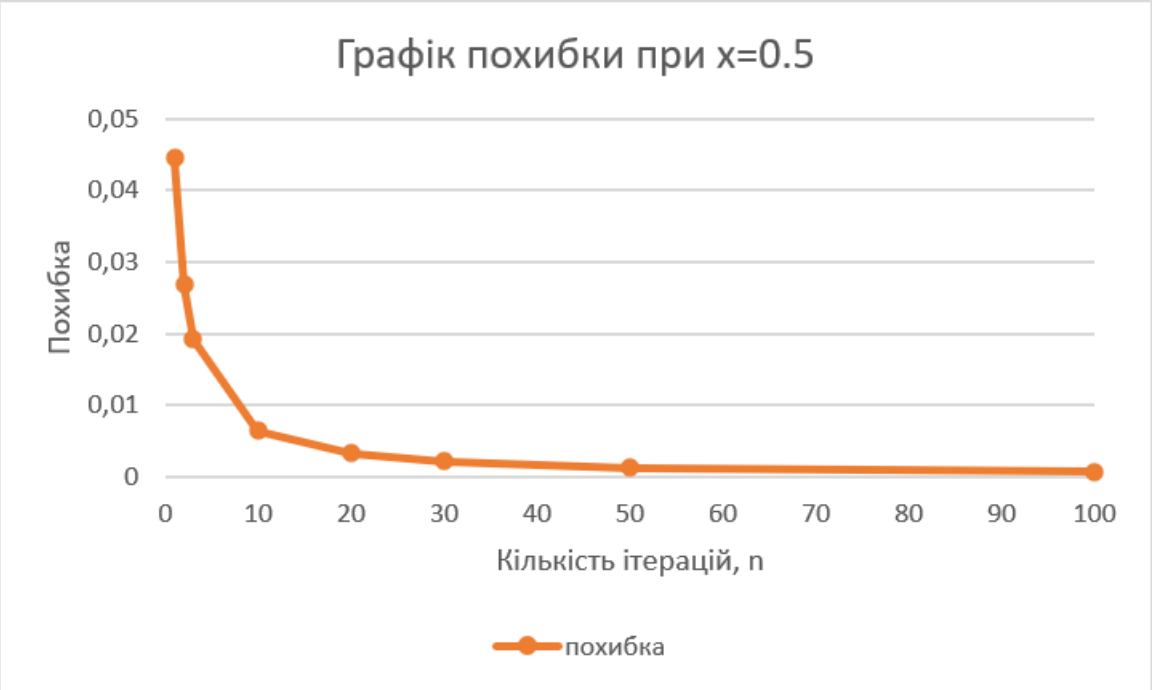
Графік залежності кількості операцій від n:



Таблиця похибки:

n	1	2	3	10	20	30	50	100
похибка	0,0444517	0,0268235	0,0191656	0,0063707	0,0032586	0,0021891	0,0013215	0,0006638

Графік залежності похибки обчислень від n:



Висновок:

Під час виконання лабораторної роботи я засвоїв теоретичний матеріал та набув практичних навичок використання різних циклічних керуючих конструкцій, вкладеній циклів, методу динамічного програмування та обчислення кількості операцій алгоритмів. Обидві програми дають однаковий результат. Із зростанням n похибка зменшується приблизно як 0.16/n, тобто добуток збігається до cos x повільно, тож для високої точності потрібне велике n. Кількість операцій у двох способів відрізняється суттєво. Це добре видно на графіку залежності кількості операцій від n. Отже, метод динамічного програмування дозволяє суттєво прискорити обчислення: кожне наступне значення береться з попереднього, а не обчислюється заново. Результат при цьому не змінюється, а код стає простішим і коротшим.